Enhancing Beef and Pork Image Classification Using Hyperparameter-Tuned ResNet50

Abstract*—In Indonesia, the adulteration of meat, particularly the blending of beef and pork, presents a significant issue, especially for the Muslim population that adheres to halal dietary requirements. Distinguishing between beef and pork by sight alone is often challenging, making it necessary to employ technology to accurately identify the types of meat. This research focused on utilizing the ResNet50 architecture to automatically differentiate images of beef and pork. The datasets included pork, meat, and horse meat. The model was developed through transfer learning, using the pre-trained weights of ResNet50 from ImageNet. ResNet50 was selected due to its ability to manage very deep networks without losing performance and its established success in various image classification tasks. The study also examined various hyperparameter combinations, such as batch size, learning rate, and dropout rate. The image dataset was split into 80% for training and 20% for testing, with 20% of the training data set aside for validation. The experimental findings revealed that the optimal performance was achieved with a batch size of 8, a learning rate of 0.0001, and a dropout rate of 0.2. Evaluation using a confusion matrix showed high performance, with an accuracy of 98%, precision of 100%, recall of 96%, and an F1-score of 98%. All evaluation metrics considered pork as the positive class. These findings suggest that the ResNet50 model is effective in distinguishing between beef and pork images and can serve as a tool to objectively confirm the halal status of meat products.

***Keywords*—image classification,ResNet50, hyperparameters, Halal, beef, pork**

I. Introduction

Meat is one of the most widely consumed animal protein sources in Indonesia. The increasing population has led to a significant rise in beef consumption, which is projected to reach 759.67 thousand tons in 2024, representing an increase of 11.71% from 2023[1]. However, growing demand has also triggered unethical practices such as meat adulteration, particularly the mixing of beef and pork. These actions not only harm consumers economically but also raise serious ethical and religious concerns, especially among Muslims who are obligated to consume halal foods[2].

Indonesia, as a Muslim-majority country, places high importance on halal food certification. The Qur'an strictly prohibits the consumption of pork, as stated in QS. Al-Baqarah:173, QS. Al-An'am:145; QS. An-Nahl:115. However, various regions have reported cases of pork being disguised as beef. For example, in East Lampung, pork was sold under the guise of beef, and in Bandung Regency, 63 tons of pork mixed with borax was packaged to resemble beef, making it difficult for the public to differentiate[3].

Visual similarities between beef and pork, including color, texture, and fat patterns, often prevent accurate identification by the general public [4]. In some instances, pork is manipulated using beef blood or additives to mimic the characteristics of beef [5]. While certain features, such as fiber structure, fat type, aroma, and texture can distinguish between meat types, these signs can be ambiguous without technical expertise.

To tackle this issue, researchers have investigated image-processing technologies to automatically distinguish meat types based on their visual features. Deep learning, especially Convolutional Neural Networks (CNNs), has shown impressive results in various image classification tasks, including the classification of meat. A study [6] achieved an accuracy of 88.75% when using CNNs to classify images of beef and pork without backgrounds. However, the accuracy significantly decreased to 73.375% when real-world backgrounds were included, revealing a lack of robustness in the model. Further research by [7] and [3] employed CNNs and AlexNet with data augmentation, achieving accuracies of 82.20% and 85%, respectively. While these findings are encouraging, they are not yet optimal, underscoring the necessity for more advanced architectures.

Transfer learning has emerged as a promising alternative, leveraging pre-trained models on large datasets to improve classification accuracy in smaller domain-specific datasets. One widely adopted architecture is ResNet50, introduced in [8], which incorporates residual learning to enable deeper and more stable networks. ResNet50 consistently outperformed the conventional CNNs in various classification tasks. For example, [9] reported 99.75% accuracy in rice leaf disease classification using ResNet50, surpassing models such as InceptionV3, VGG16, and VGG19.

This study aimed to develop an image classification model based on the ResNet50 architecture to distinguish between beef and pork images. By employing transfer learning and data augmentation, the proposed model seeks to enhance classification performance and generalizability. This approach is expected to support efforts to combat meat adulteration and ensure halal compliance in food products across Indonesia.

II. Metodhology

The study commenced with the collection of data from datasets containing pork, meat, and horse meat, but concentrated solely on images of beef and pork. This data was then divided into subsets for training, validation, and testing purposes. Pre-processing involved resizing all images and augmenting the training data to enhance generalization. The model was developed using transfer learning with the ResNet50 architecture and assessed based on accuracy, precision, recall, and F1-score, with pork designated as the positive class for all evaluation metrics. Figure 1 provides a visual representation of the overall research process. Detailed descriptions of each research step are provided below.

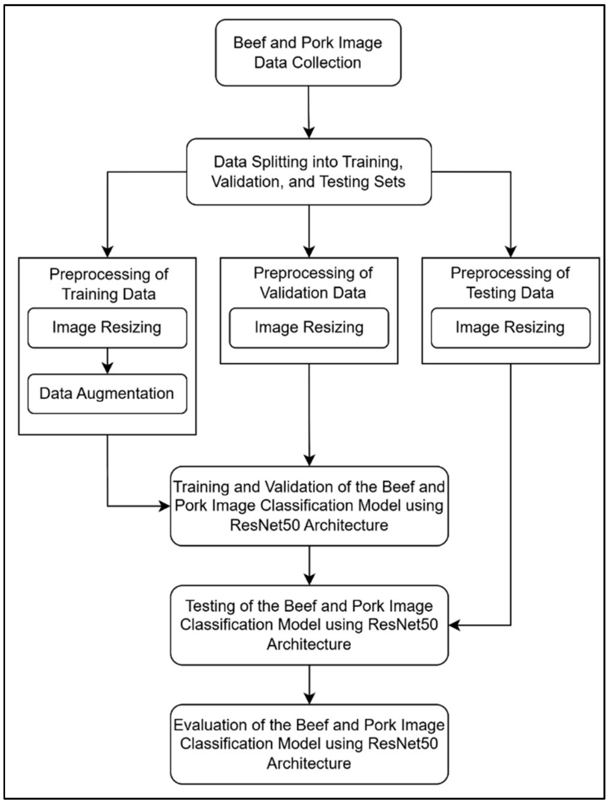
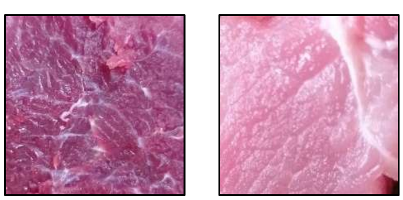


Figure 1 Research Method

A. Data Collection

The dataset used in this study was obtained from the Kaggle platform, specifically the pork, meat, and horse meat datasets. In total, 244 images were used, consisting of 115 images of beef and 129 images of pork. All images were in JPG format with varying resolutions and were in the RGB color mode.

The visual characteristics of each meat type differed significantly. Beef typically appears bright red, has a coarse texture, and contains significant intramuscular fat (marbling) with a yellowish-white color. In contrast, pork tends to appear pale pink, has a smoother texture, and its marbling is finer and more evenly distributed with white fat color. The sample images for each class are presented in Figure 2.



(a) (b)

Figure 2 Sample Images from the Dataset (a) Beef (b) Pork

B. Data Splitting

The dataset was split into two parts, allocating 80% for training and 20% for testing. Additionally, 20% of the training data was set aside for validation, following the data-partitioning method used in. The training set was employed to develop the model, the validation set was used to track performance during training, and the testing set was utilized to assess the final model. The data was randomly divided to ensure balanced class distribution. Table 1 provides the number of samples in each subset.

Table 1 Data Splitting

Class	Training Data (80%)		Test Data (20%)	Total
	Training Data (80%)	Validation Data (20%)		
Beef	72	19	24	115
Pork	82	21	26	129

C. Data Preprocessing

Data preprocessing is a crucial step in preparing the dataset for model training. The pre-processing stages in this study include image resizing and data augmentation applied to the training set.

1) Image Resizing

Due to the varying dimensions of the images in the dataset, all images were resized to 224 × 224 pixels with 3 color channels (224 × 224 × 3) to match the standard input requirements of the ResNet50 architecture. The resizing process was performed using the resize function from Keras.

2) Data Augmentation

The original training dataset showed a class imbalance, with 72 beef images and 82 pork images. To address this issue, offline augmentation was applied to the minority class (beef) to equalize the number of images in each class. The augmented images were saved as new files, resulting in a balanced training set of 82 images for each class, as shown in Table 2.

Table 2 Comparison of Training Data Quantity Before and After Augmentation

Class	Before Augmentation	After Augmentation
Beef	72	82
Pork	82	82

In addition to offline augmentation, real-time data augmentation was applied during model training using Keras ImageDataGenerator. This method introduced random transformations on each batch to increase image variation and prevent overfitting. The augmentation techniques used in this study included rotation range, width shift range, height shift range, shear range, zoom range, and horizontal flip. These transformations enhanced the diversity of the training data by introducing variations in orientation, position, and scale, thereby helping the model to generalize better and become more robust to image distortions. Examples of augmented images produced using these techniques are illustrated in Figure 3.

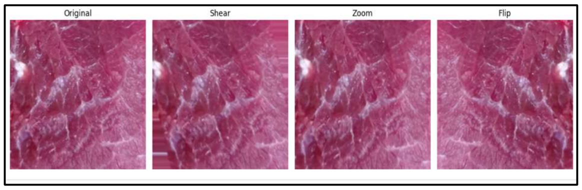


Figure 3 Original image and augmented result

D. Model Training

The model was trained using 164 images, consisting of 82 beef and 82 pork images. Each image was resized to 224×224 pixels and processed in the RGB format, resulting in input dimensions of (224, 224, and 3). Training was performed using a fully connected layer on top of a ResNet50 backbone with pre-trained weights from the ImageNet dataset. The goal of this training process was to enable the model to learn discriminative visual features from the input images for accurate classification of beef and pork.

The architecture used in this study is based on ResNet50. A notable modification of the standard ResNet50 architecture was made in the classification head, and the original fully connected layers were replaced with a custom binary classification layer. Additionally, the activation function was changed from softmax to sigmoid because the task involved binary classification. This adjustment ensures that the model outputs a single probability value suitable for distinguishing between two classes rather than a distribution across multiple categories. The architecture used in this study is illustrated in Figure 4.

Training was conducted for a maximum of 50 epochs with early stopping applied using a patience value of 5. This means that the training process would stop early if no improvement in the validation accuracy was observed over five consecutive epochs.

To optimize model performance, three key hyperparameters were tuned: batch size, learning rate, and dropout rate [3], [7], [10]. These hyperparameters were selected based on prior studies and evaluated systematically using Grid Search to find the most effective configuration for model convergence and generalization. The hyperparameter values tested in this study, along with their respective references, are summarized in Table 3.

Grid Search was employed to explore all possible combinations of the hyperparameter values listed in Table 3. This comprehensive approach enabled the identification of the best-performing configuration in terms of validation accuracy, while minimizing overfitting and improving the model's generalization on unseen data.

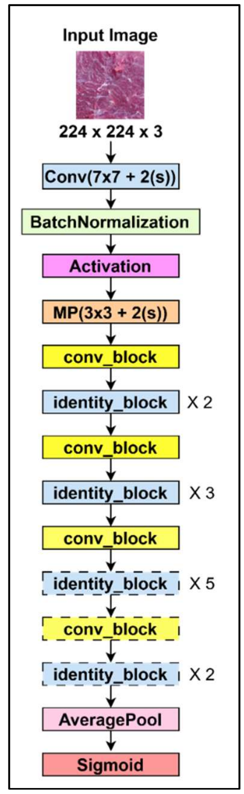


Figure 4 ResNet50 Architecture

Table 3 Hyperparameter Range for Grid Search

Hyperparameter	Value
Batch Size	[8, 16, 32]
Learning Rate	[0.01, 0.001, 0.0001]
Dropout	[0.2, 0.5, 0.7]

E. Model Testing and Evaluation

To assess the model's ability to generalize to new data, testing was performed using a dataset of beef and pork images that were not part of the training or validation phases. During this phase, all test images were resized to 224 × 224 pixels and converted to RGB format to meet the model's input specifications.

After the model was trained, it was employed to forecast the labels of the test images. These forecasts were then compared to the actual labels to evaluate the model's classification performance. This assessment involved using a confusion matrix, followed by the computation of essential performance metrics, including accuracy, precision, recall, and F1-score [3].

III. Result and Discussion

This section presents the results of training and evaluating the ResNet50 model for classifying beef and pork images using various hyperparameter configurations. A total of 27 training experiments were conducted based on the grid search method by combining three values each of batch size, learning rate, and dropout. The evaluation is divided into two parts: analysis of training results based on validation accuracy, and classification performance evaluation on the test set using confusion matrix metrics.

A. Overall Training Result

The training scenarios were crafted to identify the top-performing model by examining how hyperparameters influence validation accuracy. The training setups included batch sizes of 8, 16, and 32; learning rates of 0.01, 0.001, and 0.0001; and dropout rates of 0.2, 0.5, and 0.7. These experiments utilized grid search, and each training session incorporated early stopping with a patience of 5 epochs to avoid overfitting.

Table 4 displays the validation accuracy results from all 27 training scenarios. The configuration with a batch size of 8, a learning rate of 0.0001, and a dropout rate of 0.2 achieved the highest validation accuracy at 99.84%. In contrast, the lowest accuracy recorded was 87.85%, which occurred with a batch size of 32, a learning rate of 0.001, and a dropout rate of 0.7. These findings indicate that using smaller batch sizes and lower learning rates generally leads to improved generalization performance.

Table 4 Grid Search Result

No	Batch Size	Learning Rate	Dropout	Validation Accuracy
1	8	0.01	0.2	0.9666
2	8	0.01	0.5	0.9968
3	8	0.01	0.7	0.9666
4	8	0.001	0.2	0.9750
5	8	0.001	0.5	0.9428
6	8	0.001	0.7	0.9875
7	**8**	**0.0001**	**0.2**	**0.9984**
8	8	0.0001	0.5	0.9900
9	8	0.0001	0.7	0.9771
10	16	0.01	0.2	0.9321
11	16	0.01	0.5	0.9937
12	16	0.01	0.7	0.9937
13	16	0.001	0.2	0.9812
14	16	0.001	0.5	0.9687
15	16	0.001	0.7	0.9613
16	16	0.0001	0.2	0.9765
17	16	0.0001	0.5	0.9892
18	16	0.0001	0.7	0.9904
19	32	0.01	0.2	0.9545
20	32	0.01	0.5	0.8861
21	32	0.01	0.7	0.9642
22	32	0.001	0.2	0.9968
23	32	0.001	0.5	0.9825
24	32	0.001	0.7	0.8785
25	32	0.0001	0.2	0.9734
26	32	0.0001	0.5	0.9277
27	32	0.0001	0.7	0.9750

B. Batch Size Effect Analysis

The training results based on the batch size scenarios of 8, 16, and 32 are presented in Table 5, with Figure 5 illustrating the comparison graph of validation accuracy for each training scenario. Bold values in the table indicate the highest validation accuracy achieved.

Table 5 Validation Accuracy Results Based on Batch Size

No	Learning Rate	Dropout	8	16	32
1	0.01	0.2	0.9666	0.9321	0.9545
2	0.01	0.5	0.9968	0.9937	0.8861
3	0.01	0.7	0.9666	0.9937	0.9642
4	0.001	0.2	0.9750	0.9812	0.9968
5	0.001	0.5	0.9428	0.9687	0.9825
6	0.001	0.7	0.9875	0.9613	0.8785
7	0.0001	0.2	**0.9984**	0.9765	0.9734
8	0.0001	0.5	0.9900	0.9892	0.9277
9	0.0001	0.7	0.9771	0.9904	0.9750

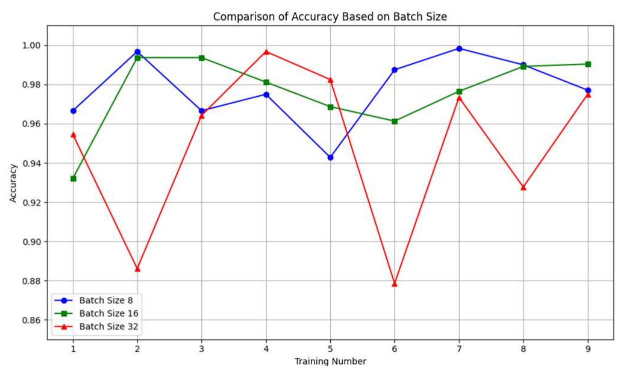


Figure 5 Comparison Validation Accuracy Based on Batch Size

Batch size is a key hyperparameter in model training, as it directly affects weight update dynamics. As shown in Table 5 and Figure 5, variations in batch size (8, 16, and 32) significantly influenced validation accuracy. Each configuration showed distinct performance characteristics, impacting both optimization and generalization.

The batch size of 8 (blue line) consistently produced the highest and most stable validation accuracy, reaching 99.84% in scenario 7 (learning rate 0.0001, dropout 0.2). Frequent weight updates and greater gradient variation helped the model adapt quickly and avoid local minima.

The batch size of 16 (green line) also showed strong and stable performance, with several combinations achieving over 99% accuracy. This batch size balanced training efficiency and model adaptability effectively.

In contrast, batch size 32 (red line) had the most inconsistent results. Although it reached 99.68% accuracy in scenario 4, it also recorded significant drops, such as 88.61%

in scenario 2. Large batches tend to produce smoother gradients, which can reduce the model's sensitivity to data variations.

In conclusion, smaller batch sizes, particularly 8, are more suitable for this dataset. Batch size 16 is a strong alternative, while batch size 32 often results in less stable and suboptimal performance.

C. Learning Rate Analysis

The training results based on learning rate scenarios of 0.01, 0.001, and 0.0001 are presented in Table 6. Figure 6 illustrates the comparison graph of validation accuracy across these scenarios. Bold values in the table indicate the highest validation accuracy obtained.

Table 6 Validation Accuracy Results Based on Learning Rate

No	Batch Size	Dropout	0.01	0.001	0.0001
1	8	0.2	0.9666	0.9750	**0.9984**
2	8	0.5	0.9968	0.9428	0.9900
3	8	0.7	0.9666	0.9875	0.9771
4	16	0.2	0.9321	0.9812	0.9765
5	16	0.5	0.9937	0.9687	0.9892
6	16	0.7	0.9937	0.9613	0.9904
7	32	0.2	0.9545	0.9968	0.9734
8	32	0.5	0.8861	0.9825	0.9277
9	32	0.7	0.9642	0.8785	0.9750

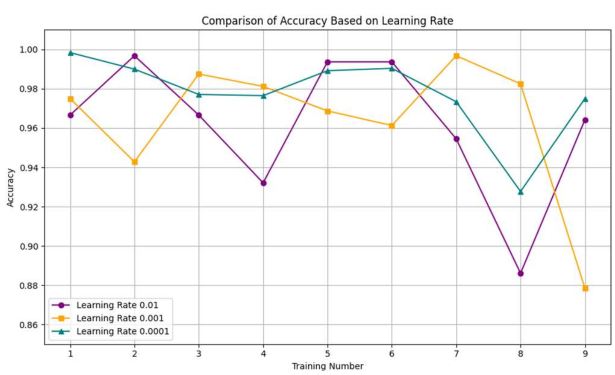


Figure 6 Comparison Validation Accuracy Based on Learning Rate

Learning rate is a crucial hyperparameter that controls the size of weight updates during training. Selecting an appropriate value ensures that the model converges effectively without overshooting or getting stuck in local minima. As shown in Table 6 and Figure 6, learning rate variations (0.01, 0.001, and 0.0001) had a significant impact on validation accuracy.

The learning rate of 0.0001 (green line) consistently achieved the best and most stable performance. The highest validation accuracy of 99.84% was reached in scenario 1 with batch size 8 and dropout 0.2. Most configurations using this rate produced accuracy above 97%, indicating its reliability in supporting smooth convergence and effective feature learning, especially on smaller datasets.

The 0.001 learning rate (orange line) delivered good results, though with greater fluctuation. Scenario 7 reached 99.68%, but others such as scenario 2 (94.28%) and scenario 9 (87.85%) showed reduced performance. This rate is a viable option but requires careful adjustment of other parameters to maintain stability.

In comparison, the 0.01 learning rate (purple line) showed the most unstable and generally lower accuracy. Although scenario 2 reached 99.68%, most combinations resulted in accuracy below 95%, with some dropping under 90%. This suggests the rate was too aggressive, disrupting the training process and leading to suboptimal outcomes.

In summary, 0.0001 is the most suitable learning rate for this dataset and model, offering stable convergence and high accuracy. Larger learning rates, especially 0.01, often reduce performance due to unstable weight updates, emphasizing the importance of careful tuning in training.

D. Dropout Analysis

The training results based on dropout rate scenarios of 0.2, 0.5, and 0.7 are presented in Table 7, while Figure 7 provides a graphical comparison of validation accuracy across the different training configurations. Bold values in the table indicate the highest validation accuracy achieved.

Table 7 Validation Accuracy Results Based on Dropout

No	Batch Size	Learning Rate	0.2	0.5	0.7
1	8	0.01	0.9666	0.9968	0.9666
2	8	0.001	0.9750	0.9428	0.9875
3	8	0.0001	**0.9984**	0.9900	0.9771
4	16	0.01	0.9321	0.9937	0.9937
5	16	0.001	0.9812	0.9687	0.9613
6	16	0.0001	0.9765	0.9892	0.9904
7	32	0.01	0.9545	0.8861	0.9642
8	32	0.001	0.9968	0.9825	0.8785
9	32	0.0001	0.9734	0.9277	0.9750

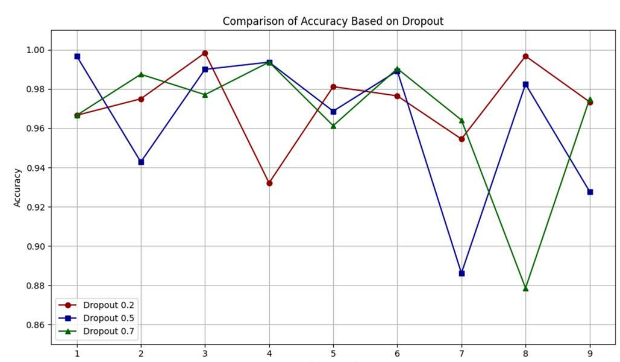


Figure 7 Comparison Validation Accuracy Based on Dropout

Dropout is a regularization method that mitigates overfitting by randomly turning off neurons during the training process. Selecting the right dropout rate is crucial to strike a balance between regularization and the model's ability to learn. As illustrated in Table 7 and Figure 7, different dropout rates (0.2, 0.5, and 0.7) significantly affected validation accuracy.

A dropout rate of 0.2 (maroon line) produced the best and most stable results. The highest accuracy of 99.84% was achieved in scenario 3 (batch size 8. learning rate 0.0001). while other combinations. such as scenario 8. reached 99.68%. This dropout rate applies light regularization. allowing most neurons to remain active and enabling the model to effectively capture complex patterns. which is beneficial for small datasets.

The dropout rate of 0.5 (dark blue line) also yielded strong performance. though with more variation. High accuracies were recorded in scenario 1 (99.68%) and scenario 4 (99.37%). However. accuracy dropped significantly in scenario 7 to 88.61%. indicating that stronger regularization may interfere with learning if not properly tuned.

In contrast. a dropout rate of 0.7 (dark green line) resulted in the least stable and generally lower accuracy. The best result was 99.37% in scenario 4. but most outcomes were below 97%. including 87.85% in scenario 8. This rate likely deactivates too many neurons. limiting the model's capacity to learn meaningful features.

In conclusion. a dropout rate between 0.2 and 0.5 is most suitable for this study. Dropout 0.2 works well with smaller batch sizes and lower learning rates. while dropout 0.5 can be used with higher learning rates to control overfitting. Dropout 0.7 should be avoided due to its tendency to reduce model performance and stability.

E. *Final Model Evaluation*

The optimal model was achieved by utilizing a combination of a batch size of 8, a learning rate of 0.0001, and a dropout rate of 0.2. This setup consistently reached peak accuracy during both training and validation phases. A batch size of 8 facilitated efficient training with stable gradients, while the low learning rate of 0.0001 ensured smooth convergence. The dropout rate of 0.2 effectively minimized overfitting without sacrificing crucial information. This combination led to a model that was both robust and well-generalized. The model's performance was further assessed using a confusion matrix, as depicted in Figure 8.

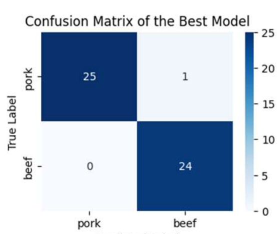


Figure 8 Confusion Matrix of th Best Model

Figure 8 presents the confusion matrix for the top-performing model evaluated on 50 images. The results included 25 true positives (TP), 24 true negatives (TN), 0 false positives (FP), and 1 false negative (FN), with pork designated as the positive class. From these findings, the model achieved a precision of 1.00, signifying that all predicted pork images were accurate. The recall was 0.96, indicating the model successfully identified 96% of the actual pork images. The overall accuracy stood at 0.98, with 49 out of 50 predictions being correct. The F1-score was 0.98, demonstrating a strong equilibrium between precision and recall.

IV. CONCLUSION

This study successfully implemented the ResNet50 architecture to effectively classify images of beef and pork. Based on the research question regarding the performance of the ResNet50 architecture in distinguishing between the two types of meat images. the results demonstrate that utilizing transfer learning from a pre-trained model significantly improves classification accuracy. The image data were split into 80% training and 20% testing. with 20% of the training data used as validation. Experimental results showed that the model achieved its best performance with a combination of hyperparameters consisting of a batch size of 8. a learning rate of 0.0001. and a dropout rate of 0.2. Evaluation using a confusion matrix yielded an accuracy of 98%. precision of 100%. recall of 96%. and F1-score of 98%. These results indicate that the model has excellent capability in distinguishing between beef and pork images. with a very low misclassification rate and high prediction accuracy.

REFERENCES

[1] Badan Pusat Statistik. *Peternakan dalam Angka 2024*. vol. 9. Jakarta: Badan Pusat Statistik. 2024. Accessed: Mar. 22. 2025. [Online]. Available: https://www.bps.go.id

[2] I. Saraswati. R. Fahrizal. A. Nuur Fauzan. and M. Ali Setyo Yudono. "Klasifikasi Daging Sapi. Kambing. dan Babi menggunakan Jaringan Saraf Tiruan Perambatan Balik Berbasis Tekstur GLCM." *Sistemasi: Jurnal Sistem Informasi*. vol. 13. no. 6. pp. 2698–2708. 2024. [Online]. Available: http://sistemasi.ftik.unisi.ac.id

[3] I. Akhmad DLY. J. Jasril. S. Sanjaya. L. Handayani. and F. Yanto. "Klasifikasi Citra Daging Sapi dan Babi Menggunakan CNN Alexnet dan Augmentasi Data." *Journal of Information System Research (JOSH)*. vol. 4. no. 4. pp. 1176–1185. Jul. 2023. doi: 10.47065/josh.v4i4.3702.

[4] A. H. Artya. J. Jasril. S. Sanjaya. F. Syafria. and E. Budianita. "Implementasi Convolutional Neural Network Untuk Klasifikasi Daging Menggunakan Fitur Ekstraksi Tekstur dan Arsitektur AlexNet." *JURIKOM (Jurnal Riset Komputer)*. vol. 9. no. 3. pp. 635–643. Jun. 2022. doi: 10.30865/jurikom.v9i3.4177.

[5] L. Handayani. "Analisa Metode Gabor dan Propbabilistic Neural Network untuk Klasifikasi Citra (Studi Kasus: Citra Daging Sapi dan Babi)." *Jurnal Sains. Teknologi dan Industri*. vol. 14. no. 2. pp. 169–177. Jun. 2017. [Online]. Available: http://ejournal.uin-suska.ac.id/index.php/sitekin

[6] R. H. Laluma. B. Sugiarto. A. Santriyana. A. G. Azwar. N. Nurwathi. and G. Gunawan. "Klasifikasi Perbedaan Daging Sapi dan Daging Babi dengan Metode Convolutional Neural Network Berbasis Web." *Infotronik : Jurnal Teknologi Informasi dan Elektronika*. vol. 6. no. 1. Jun. 2021. doi: 10.32897/infotronik.2021.6.1.603.

[7] I. M. A. M. Dinata. I. G. A. Gunadi. and I. M. G. Sunarya. "Analisis Hyperparameter Pada Klasifikasi Jenis Daging Menggunakan Algoritma Convolutional Neural Network." *Jurnal Sains*

Komputer & Informatika (J-SAKTI. vol. 8. no. 1. pp. 25–34. Mar. 2024.

[8] K. He. X. Zhang. S. Ren. and J. Sun. “Deep Residual Learning for Image Recognition.” Dec. 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[9] S. R. Shah. S. Qadri. H. Bibi. S. M. W. Shah. M. I. Sharif. and F. Marinello. “Comparing Inception V3. VGG 16. VGG 19. CNN. and ResNet 50: A Case Study on Early Detection of a Rice Disease.” *Agronomy*. vol. 13. no. 6. Jun. 2023. doi: 10.3390/agronomy13061633.

[10] M. F. Martias. J. Jasril. S. Sanjaya. L. Handayani. and F. Yanto. “Klasifikasi Citra Daging Sapi dan Daging Babi Menggunakan CNN Arsitektur EfficientNet-B6 dan Augmentasi Data.” *Jurnal Sistem Komputer dan Informatika (JSON)*. vol. 4. no. 4. p. 642. Jun. 2023. doi: 10.30865/json.v4i4.6195.